

COET TIMETABLE MANAGEMENT, GROUP ALLOCATION AND VALIDATION SYSTEM



Implementation Report **Tutorial, Seminar, and Practical Group Allocation**

DATE: 26TH SEP, 2026

prep by
EMMANUEL CHARLES
KODIN SOFTWARES



1. Purpose

Add a coordinator-controlled feature that allocates student groups to existing **seminar, tutorial, and non-workshop practical sessions**. The system must use each course's configured activity requirements, avoid timetable conflicts and venue-capacity violations, and try alternate assignments when later requirements cannot be completed.

Workshops are not allocated by this feature. They still count when checking group availability.

2. Confirmed requirements

- A course may require a seminar, tutorial, practical, or a combination such as **Tutorial + Practical**.
- Every group studying a course must complete each activity configured for that course.
- A course code has one shared set of requirements across all programmes.
- Groups count as **30 students** for capacity calculations in the first version.
- Missing or zero-capacity venues are unresolved issues. The coordinator must correct them; the allocator must not assume those venues have capacity.
- There is no special minimum capacity for seminar venues beyond the normal assigned-student capacity check.
- Courses with blank requirements mean **no allocation required**, but must be flagged for coordinator attention.
- Groups from different programmes may share a session if they study the same course and all constraints are met.
- The allocator may reassign existing links by default. It must report changes and provide a way to revert them.
- Allocation priority is **seminar** → **tutorial** → **practical**. If later work cannot be completed, the engine must revisit earlier choices and try alternatives.

3. Existing project foundation

The project already has:

- `ProgrammeCourse` rows mapping programmes to course codes and semester numbers.
- `StudentGroup` rows connected to programmes.
- `Session` rows with semester, course code, activity type, day, time, and venue.
- `SessionGroup` links connecting groups to sessions.

- Venue capacity.
- Existing group timetable collection and exports that include assigned sessions.
- Separate workshop and technical drawing allocation data that can be included in availability checks.
- An existing programme-course importer, activity logging, and hand-built Django views and templates.

The main structural gap is that course name and code currently live on each programme-course row. A shared course record is needed to make a course's requirements consistent across programmes.

4. Course data model

Shared `Course` model

Create one shared `Course` record per unique course code. It stores:

- Unique course code.
- Course name.
- Required activities, selected from Seminar, Tutorial, and Practical.

The requirements should be stored as validated values, supporting combinations. A blank requirement means the course has no allocation requirement, and the allocation review will flag it.

`ProgrammeCourse` relationship

Update `ProgrammeCourse` to reference the shared `Course`, while retaining its programme and semester number. This expresses that a programme studies a course in a semester, while the course's requirements remain global.

Before migrating existing data, identify same-code rows that have conflicting course names. Report these for review rather than silently selecting a name.

Session records

`Session` currently stores `course_code` as text. In the first implementation, it can remain that way; the allocator matches sessions to `Course` by normalized course code. A `Course` foreign key on `Session` is a possible later improvement, but is not required to implement allocation.

5. Import behavior

No separate importer or import page is needed. Update the existing programme-course import so it also populates the shared `Course` model.

Add an optional workbook column such as **Required Activities**, accepting clear aliases such as `required activities`, `allocation requirements`, and `activities required`. Values should support combinations such as `Tutorial; Practical`.

For each imported programme-course row, the existing import should:

1. Find or create the shared `Course` by course code.
2. Validate and set its course name and required activities.
3. Create or update the programme's `ProgrammeCourse` link, including semester number.
4. Report invalid activity values or conflicting course names as import issues.

Older workbooks without the new column should still import. Their courses will have blank requirements and be flagged during allocation review.

If the same course code appears more than once in one workbook with conflicting names or requirements, the importer should report a conflict instead of silently choosing a value. Re-importing should remain idempotent and follow the existing importer's reconciliation behavior.

6. Allocation eligibility and completeness

For the selected semester, the system determines group requirements by:

1. Finding the courses studied by each group's programme in that semester.
2. Reading the required activities from each shared `Course`.
3. Finding existing sessions for those course codes, semester, and required activity types.
4. Assigning every group studying the course to the activity sessions required by the course.

Courses with blank requirements are not allocated, but appear in a "No allocation required — requirement not configured" notice. A configured requirement with no matching session is reported as unresolved, identifying the course, activity, and affected groups.

Initial assumption: each configured activity is required once per group per course. If some courses require multiple tutorials or practicals, the requirement data must include a count.

7. Validation

Use the same validation logic for automatic and manual assignments.

Course eligibility

A group may only be assigned to a session if its programme studies that course in the selected semester and the session's activity is required for that course.

Timetable availability

A group cannot be assigned to overlapping activities on the same day. Check against:

- Existing assigned sessions, including lectures.
- Proposed assignments from the current allocation run.
- Workshop allocations.
- Technical drawing allocations.

For clock-based activities, use interval overlap: the candidate starts before the existing activity ends and ends after the existing activity starts.

Some imported workshops have period information but no clock times. Reuse the project's workshop time-resolution logic where possible. If a workshop's time cannot be determined, report that availability could not be verified.

Venue capacity

- Use 30 students per group.
- Required capacity is $\text{number of assigned groups} \times 30$.
- A missing venue or zero/unknown venue capacity cannot pass validation.
- Leave affected assignments unresolved and explain that the coordinator must correct the venue.
- Do not apply a special minimum capacity for seminars.

Other invalid data

Report invalid session times and other data that prevents reliable validation. Do not silently accept assignments that cannot be checked.

8. Allocation engine and backtracking

Put the scheduling logic in a dedicated module such as `core/group_allocation.py`, separate from the Django views.

Priority and search

Run the stages in this order:

1. Seminar
2. Tutorial
3. Practical

The order prioritizes the search; it does **not** make earlier assignments permanent. If tutorials or practicals cannot be completed, the engine must backtrack and try alternative seminar or tutorial choices before concluding that it could not find a complete allocation.

Within each stage, the engine should:

- Start with the most constrained group-course requirements, such as those with the fewest eligible sessions.
- Consider eligible sessions and check conflicts and capacity.
- Rank valid sessions according to the soft preferences below.
- Try alternate candidates when later assignments fail.
- Return unresolved requirements with specific reasons if it cannot find a valid complete allocation.

Use deterministic candidate ordering so identical input data produces repeatable results. If a practical search limit is reached, report that the search limit was reached; do not claim that a valid allocation is impossible without having established that.

Soft preferences

After all hard constraints pass, prefer:

1. Keeping existing valid assignments.
2. Balancing group counts across sessions.
3. Avoiding unnecessary timetable gaps and overly concentrated days.
4. Combining eligible groups from different programmes in the same session.

These preferences must never override eligibility, capacity, or timetable availability.

9. Coordinator workflow and revert

Recommended workflow:

1. Coordinator selects a semester and chooses Seminar, Tutorial, Practical, or all configured activities.
2. System flags courses with blank requirements and checks whether required sessions exist.
3. Engine calculates assignments, revisiting earlier choices when needed.
4. Coordinator reviews proposed assignments, changes, warnings, and unresolved items.
5. Valid assignments are applied together in a database transaction.
6. System summarizes assignments added, moved, retained, or unresolved.

7. Coordinator can revert that run.

Save each applied run and the relevant before-and-after assignments so revert remains possible after the request ends. Before reverting, check that later edits have not changed the assignments involved; if they have, report the conflict instead of overwriting those edits.

Manual assignment must use the same validator. This keeps coordinator edits consistent with automatic allocation rules.

Workflow assumption: valid assignments can be applied even when some items remain unresolved, with those items left for coordinator review. If the coordinator should instead be required to resolve every issue before any changes are applied, that should be specified before implementation.

10. Coordinator interface and audit

Add an allocation page using the project's existing navigation and template conventions. It should provide:

- Semester selector and activity scope.
- Allocation action.
- Results organized by activity and status.
- Proposed group, programme, course, session, time, venue, assigned group count, and capacity status.
- Warnings for blank requirements and missing required sessions.
- Specific reasons for unresolved assignments.
- Apply and revert actions.
- Manual assignment controls with validation feedback.

Log applied changes through the existing `ActivityLog` helper using assignment and removal actions. Keep detailed before-and-after history in the allocation-run records so the coordinator can review and revert a run.

11. Timetable exports

Apply assignments by creating or updating `SessionGroup` links. Existing group timetable collection and exports already consume those links, so allocated tutorials, seminars, and practicals should appear in group timetables without a change to the export format.

Workshops continue to use their existing allocation and display paths.

12. Implementation sequence

1. Review and resolve inconsistent existing names for identical course codes.
2. Add the shared `Course` model and migrate current `ProgrammeCourse` data to it.
3. Update course and programme-course forms and views for the new relationship.
4. Update the existing programme-course import to read and validate Required Activities.
5. Implement reusable course-eligibility, timetable-availability, and venue-capacity validators.
6. Implement the preview and backtracking allocator.
7. Add allocation-run persistence, transactional apply, logging, and guarded revert.
8. Add the coordinator page and validated manual assignment workflow.
9. Confirm applied assignments appear in existing group timetable exports.

13. Assumptions to verify during implementation

- Each configured activity is required once per group per course.
- Valid assignments may be applied while unresolved items remain for coordinator review.
- Existing course-code comparisons need consistent normalization, such as trimming whitespace and matching case consistently, before shared courses are migrated or imported.